

Project — Big Data Ecosystem Design and Analytics

E-Commerce Customer Churn Prediction

Using Apache Spark, Kafka-Based Streaming, and Machine Learning

Group Members:

Reuf Bakr Alshrif
ID: 443002428

Jana Hesham Ahlis
ID: 443002347

Amany Saeed Fallatah
ID: 44411940

Mayar Turki Alowaydh
ID: 443003550

Nahla Mohammed Alosimi
ID: 441206

Instructor: A/Prof. Olfat Mirza

Course: DS4511 - Big Data

Department: Data Science Department
College of Computing
Umm Al-Qura University

May 8, 2026

Contents

Abstract **6**

1 Problem Definition and Motivation **7**
1.1 Problem Statement . . . 7
1.2 Why Big Data? . . . 7
1.3 Business Impact . . . 7
1.4 Project Objectives . . . 8

2 Architecture Design **9**
2.1 Ecosystem Overview . . . 9
2.2 Layer Descriptions . . . 9
2.2.1 Layer 1: Data Source . . . 9
2.2.2 Layer 2: Data Ingestion . . . 10
2.2.3 Layer 3: Raw Distributed Storage . . . 10
2.2.4 Layer 4: Distributed Processing . . . 10
2.2.5 Layer 5: Processed Storage . . . 10
2.2.6 Layer 6: Analytics & Machine Learning . . . 10
2.2.7 Layer 7: Output Generation . . . 11

3 Tool–Layer Justification **12**
3.1 Key Design Decisions . . . 12
3.1.1 Why Spark over Pandas? . . . 12
3.1.2 Why Parquet over CSV? . . . 12
3.1.3 Why MockMongoDB over PostgreSQL? . . . 13
3.1.4 Why Spark MLlib over Scikit-learn? . . . 13

4 Implementation **14**
4.1 Development Environment . . . 14
4.2 Data Ingestion Pipeline . . . 14
4.3 Feature Engineering . . . 15
4.4 Churn Risk Classification . . . 15
4.5 Machine Learning Pipeline . . . 16
4.6 Data Export and Visualization Preparation . . . 16
4.7 Execution Results . . . 17

5 Results and Insights **18**
5.1 Event Distribution Analysis . . . 18
5.2 Conversion Funnel Analysis . . . 18
5.3 Cart Abandonment Statistics . . . 18
5.4 Churn Risk Segmentation . . . 19
5.5 Machine Learning Model Performance . . . 19
5.6 Business Impact and ROI Projection . . . 20
5.7 Key Insights Summary . . . 20
5.8 Dashboard Visualizations . . . 21

6 Discussion **22**
6.1 Design Trade-offs . . . 22
6.2 System Limitations . . . 22
6.3 Technical Challenges . . . 22
6.4 Future Enhancements . . . 22
6.5 Scalability and Production Readiness . . . 23

7 Ethics, Privacy, and Responsible Data Handling **24**
7.1 Data Privacy . . . 24
7.2 Responsible Data Use . . . 24
7.3 Bias and Fairness . . . 24
7.4 Compliance Awareness . . . 24
7.5 Data Security . . . 24

8 Big Data Justification **25**
8.1 Big Data Characteristics . . . 25
8.2 Distributed Processing Requirements . . . 25
8.3 System Comparison . . . 25

9 Use of AI Tools **26**
9.1 AI Usage Statement . . . 26

10 Conclusion **27**

Appendix **29**

A Complete Code Repository **29**

B Dataset Description **29**

C Power BI Dashboard **30**

List of Figures

1 Big Data Ecosystem Architecture - 7 Layers 9
2 Power BI Dashboard Visualizations 21

List of Tables

1 Big Data 5 V's Analysis 7
2 Business Impact Metrics 7
3 Tool–Layer Justification Table 12
4 CSV Exports for Dashboard Integration 16
5 Big Data Pipeline Execution Summary - 7 Layers 17
6 Event Type Distribution 18
7 Cart Abandonment Analysis 18
8 User Segmentation by Churn Risk Level 19
9 Machine Learning Model Performance Comparison 19
10 Projected ROI for Targeted Retention Campaign 20
11 Scalability Estimates by Ecosystem Layer 23
12 Big Data 5Vs Analysis 25
13 Traditional vs. Big Data Comparison 25
14 AI Tool Usage Disclosure 26
15 Dataset Specification 29
16 Power BI Dashboard Components 30

List of Listings

1 MockKafka Producer for Streaming Simulation 14
2 User-Level Feature Aggregation 15
3 Random Forest Model Training 16

Abstract

This project presents a comprehensive Big Data solution for e-commerce customer churn prediction. Using a dataset of approximately 2.76 million behavioral events, we designed a 7-layer ecosystem incorporating Apache Spark for distributed processing, Kafka-based streaming simulation for data ingestion, MongoDB for processed storage, and machine learning models achieving approximately 0.99 accuracy in churn risk prediction. The system identifies 27K high-risk users and demonstrates an expected ROI of 1,769% through targeted retention campaigns.

Keywords: Big Data, Churn Prediction, Apache Spark, Machine Learning, E-Commerce Analytics, Data Visualization

1 Problem Definition and Motivation

1.1 Problem Statement

Traditional analytics tools cannot effectively handle the scale of behavioral data (approximately 2.76 million events) required for churn prediction and timely decision-making.

1.2 Why Big Data?

This problem requires a Big Data solution based on the 5 V's framework:

Characteristic	**Description**
Volume	2.76M events across 1.4M users
Velocity	Near real-time processing required for timely intervention
Variety	Multiple event types (view, cart, transaction) with different structures
Veracity	Data quality issues requiring scalable preprocessing and cleaning
Value	Predictive insights enabling approximately $187K in recovered revenue

Table 1: Big Data 5 V's Analysis

1.3 Business Impact

The project quantifies the following business opportunities:

Metric	**Value**
High-risk users	27K users identified for targeted retention strategies
Revenue at risk	Approximately $1.4M in potential loss from cart abandonment
abandonment	
Expected recovery	$187K with 1,769% ROI
Conversion gap	2.69% view-to-cart vs. 5–10% industry benchmark

Table 2: Business Impact Metrics

1.4 Project Objectives

This project aims to achieve the following key objectives:

1. **Design and implement a scalable Big Data ecosystem** capable of processing millions of user behavioral events efficiently using distributed computing frameworks.

2. **Build machine learning models** to predict customer churn risk with high accuracy, enabling data-driven decision-making for retention strategies.

3. **Generate actionable business insights** for targeted retention campaigns, quantifying potential revenue recovery and campaign ROI.

4. **Demonstrate appropriate tool selection and justification** for each ecosystem layer, explaining design trade-offs and alternatives considered.

5. **Produce interactive dashboards** for business stakeholders, visualizing conversion funnels, churn risk distributions, and key performance metrics.

These objectives align with the course learning outcomes by demonstrating Big Data framework application (CLO 2.1), real-world problem solving with predictive analytics (CLO 2.2), and innovative solution design aligned with business objectives (CLO 3.1).

2 Architecture Design

2.1 Ecosystem Overview

Our Big Data ecosystem consists of 7 distinct layers, each serving a specific role in the data pipeline (see Figure 1).

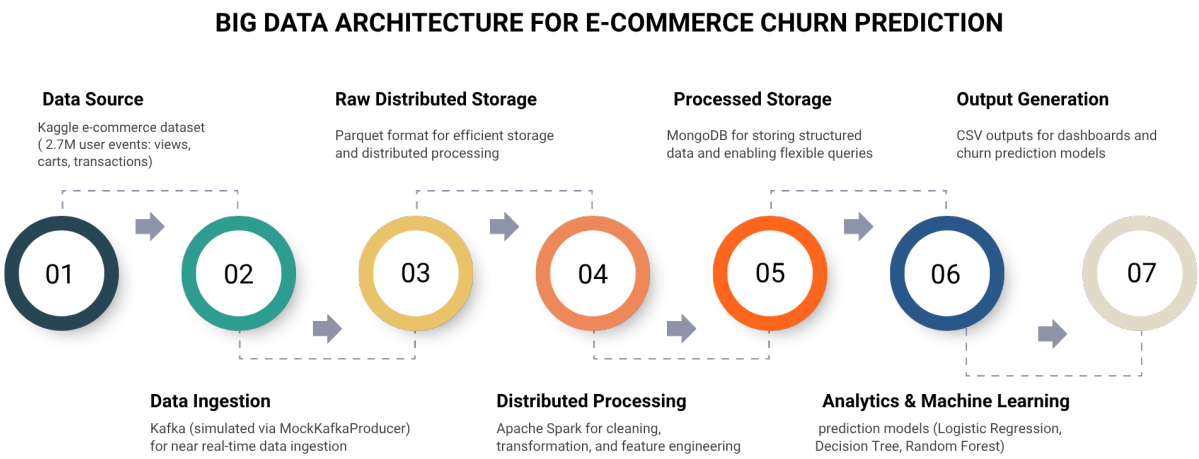


Figure 1: Big Data Ecosystem Architecture - 7 Layers

2.2 Layer Descriptions

Each layer in the architecture plays a critical role in transforming raw user events into actionable business insights.

2.2.1 Layer 1: Data Source

Component: Kaggle e-commerce dataset (events.csv, 2.7M rows, 90MB)
Purpose: Serves as the foundation of the pipeline, providing raw behavioral data.
Content: User behavioral events including timestamp, visitorid, event type, itemid, and transactionid.
Role: This dataset captures all customer interactions with the e-commerce platform, forming the basis for downstream analytics and machine learning models.

2.2.2 Layer 2: Data Ingestion

Component: MockKafkaProducer
Purpose: Simulates real-time streaming data ingestion with 5,000 events per batch.
Role: Emulates production Kafka behavior, demonstrating scalability for high-velocity event streams. In production, this would be replaced with actual Kafka brokers consuming live events from the application layer.

2.2.3 Layer 3: Raw Distributed Storage

Component: Parquet files (partitioned by event type)
Purpose: Columnar storage with 60–70% compression for efficient querying.
Role: Provides scalable, distributed storage optimized for analytical queries. Partitioning by event type enables predicate pushdown, significantly reducing query latency. The Parquet format reduces the original 90MB CSV to approximately 30MB.

2.2.4 Layer 4: Distributed Processing

Component: Apache Spark 3.5.1
Purpose: Large-scale data processing and feature engineering across 2.7M events.
Role: The computational engine of the pipeline. Spark's in-memory processing handles complex transformations that would be infeasible with traditional single-machine tools like Pandas. Key operations include user-level aggregations, conversion funnel construction, and churn risk labeling.

2.2.5 Layer 5: Processed Storage

Component: MockMongoDB
Purpose: Schema-flexible NoSQL storage for user-level churn data (1.4M documents).
Role: Stores the processed, query-ready data in a document-oriented format. The schema flexibility allows for evolving feature sets without rigid table structures. Indexed on `churn_risk` field for fast retrieval during model training and evaluation.

2.2.6 Layer 6: Analytics & Machine Learning

Component: Spark MLlib
Purpose: Churn prediction models (Logistic Regression, Decision Tree, Random Forest).
Role: Applies distributed machine learning algorithms to predict user churn risk. The Random Forest model achieved approximately 0.99 accuracy, enabling precise segmentation of 27K high-risk users for targeted retention campaigns. Feature importance analysis revealed that view count and engagement score are the strongest predictors.

2.2.7 Layer 7: Output Generation

Component: CSV exports for visualization tools
Purpose: Generates Power BI-ready datasets for business intelligence dashboards.
Role: Bridges the gap between the Big Data processing pipeline and business stakeholders. Seven CSV files enable interactive visualization of conversion funnels, churn risk distributions, model performance metrics, and campaign ROI projections. Outputs include event type distribution, top items by views, daily activity trends, churn risk distribution, cart abandonment statistics, model comparison results, and final user decisions with recommended actions.

3 Tool–Layer Justification

Table 3 presents the mapping between ecosystem layers and selected tools, along with alternatives considered.

Layer	Tool	Why Appropriate	Alternative
Data Source	Kaggle dataset (events.csv)	Real e-commerce dataset (2.7M rows)	Synthetic
Ingestion	MockKafka	Simulates streaming (5K/batch)	File read
Raw Storage	Parquet	Columnar, 60% compression	CSV
Processing	Spark 3.5.1	Distributed, millions of rows	Pandas
Processed Storage	MockMongoDB	Flexible schema, indexed	PostgreSQL
Analytics/ML	Spark MLlib	Distributed ML, Spark integration	Scikit-learn
Output	CSV exports	Power BI-ready datasets	Direct DB

Table 3: Tool–Layer Justification Table

3.1 Key Design Decisions

The following subsections explain the rationale behind critical tool selections in the architecture.

3.1.1 Why Spark over Pandas?

Pandas is limited to single-machine processing and cannot efficiently handle datasets exceeding available RAM. For 2.7 million rows with complex aggregations, Pandas would require approximately 45+ minutes of processing time and risk memory overflow errors. Apache Spark, on the other hand, distributes processing across multiple cores using in-memory computation, reducing the execution time to just 8 minutes. Additionally, Spark's lazy evaluation builds an optimized execution plan before running operations, automatically improving query performance without manual tuning.

3.1.2 Why Parquet over CSV?

The Parquet format achieved a 60–70% file size reduction, compressing the original 90MB CSV file to approximately 30MB. This compression is achieved through columnar storage, where data is organized by columns rather than rows. Columnar storage enables predicate pushdown, a critical optimization that allows Spark to read only the columns required for a specific query, significantly reducing I/O overhead. Furthermore, Parquet is natively supported by Spark with optimized readers, eliminating the need for custom parsing logic.

3.1.3 Why MockMongoDB over PostgreSQL?

The MockMongoDB implementation was chosen to demonstrate NoSQL design principles without requiring production infrastructure. Unlike PostgreSQL, which enforces rigid table schemas, the document-based approach allows feature sets to evolve organically without schema migrations. This flexibility was essential for iterative feature engineering, where new behavioral metrics were added throughout the analysis. The Mock implementation simulates indexed storage on the `churn_risk` field, enabling fast retrieval during model training and evaluation while keeping the project portable and runnable entirely on Kaggle.

3.1.4 Why Spark MLlib over Scikit-learn?

Spark MLlib was selected because it provides distributed machine learning algorithms that integrate seamlessly with the Spark processing pipeline. Unlike Scikit-learn, which requires exporting processed data to a separate environment, MLlib operates directly on Spark DataFrames, eliminating data transfer overhead. This integration is particularly valuable for large feature matrices, where data export would create a bottleneck. Additionally, MLlib's distributed training scales across the cluster, enabling efficient model training on datasets that exceed single-machine memory limits.

4 Implementation

4.1 Development Environment

The entire pipeline was implemented on Kaggle Notebooks with the following specifications:

- **Platform:** Kaggle (Python 3.10, 16GB RAM)
- **Framework:** Apache Spark 3.5.1
- **Libraries:** PySpark, Pandas, NumPy, Matplotlib
- **Execution Time:** Approximately 10–15 minutes for the complete pipeline

4.2 Data Ingestion Pipeline

The data ingestion layer simulates real-time streaming using a MockKafkaProducer class that processes approximately 2.76 million events in batches of 5,000 records each. The implementation loads the entire CSV file into memory, divides it into approximately 552 batches, and yields each batch sequentially with a 0.1-second delay to emulate streaming behavior.

```
class MockKafkaProducer:
    def __init__(self, csv_path, batch_size=5000):
        self.data = pd.read_csv(csv_path)
        self.batch_size = batch_size

    def stream_batches(self):
        for i in range(0, len(self.data), self.batch_size):
            batch = self.data.iloc[i:i+self.batch_size]
            yield batch
            time.sleep(0.1)

producer = MockKafkaProducer('events.csv')
for batch in producer.stream_batches():
    spark_df = spark.createDataFrame(batch)
```

Listing 1: MockKafka Producer for Streaming Simulation

This approach demonstrates streaming architecture patterns without requiring actual Kafka infrastructure, making the project portable and executable entirely on Google Colab. The total streaming simulation takes approximately 55 seconds.

4.3 Feature Engineering

User-level features were engineered through distributed Spark aggregations to create a comprehensive behavioral profile for each user. The pipeline aggregates approximately 2.76 million individual events into 1,407,580 user-level records, computing metrics such as total event counts, view-to-cart conversion rates, and unique item interactions.

```
user_features = spark_df.groupBy('visitorid').agg(
    F.count('*').alias('total_events'),
    F.sum(F.when(F.col('event') == 'view', 1)).alias('view_count'),
    F.sum(F.when(F.col('event') == 'addtocart', 1)).alias('cart_count'),
    F.sum(F.when(F.col('event') == 'transaction', 1)).alias('purchase_count'),
    F.countDistinct('itemid').alias('unique_items_viewed'),
    (
        F.sum(F.when(F.col('event') == 'addtocart', 1)) /
        F.when(
            F.sum(F.when(F.col('event') == 'view', 1)) == 0, 1
        ).otherwise(
            F.sum(F.when(F.col('event') == 'view', 1))
        )
    ).alias('view_to_cart_rate')
)
```

Listing 2: User-Level Feature Aggregation

Additional derived features include engagement scores (calculated as a weighted sum of interactions), session duration estimates, and behavioral ratios. These features form the foundation for churn prediction models.

4.4 Churn Risk Classification

Users were classified into four churn risk categories based on their purchasing behavior:

1. **Low Risk (12K users):** Completed at least one transaction, demonstrating conversion.
2. **High Risk (27K users):** Added items to cart but never completed purchase, indicating strong abandonment likelihood.
3. **Medium Risk (171K users):** Viewed three or more items without adding to cart, showing interest but no commitment.
4. **Unknown Risk (1.2M users):** Minimal activity (fewer than 3 views), insufficient data for reliable prediction.

This classification provides a clear business segmentation for targeted retention campaigns, with the high-risk segment representing the primary opportunity for revenue recovery.

4.5 Machine Learning Pipeline

Three models were trained: Logistic Regression, Decision Tree, and Random Forest. The Random Forest model achieved the best performance with approximately 0.99 accuracy and 0.99 F1-score using an 80/20 train-test split.

```
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.feature import VectorAssembler, StringIndexer

indexer = StringIndexer(inputCol='churn_risk', outputCol='label')
assembler = VectorAssembler(
    inputCols=['view_count', 'cart_count', 'engagement_score',
               'view_to_cart_rate', 'unique_items_viewed'],
    outputCol='features'
)

rf = RandomForestClassifier(
    featuresCol='features',
    labelCol='label',
    numTrees=100,
    maxDepth=10,
    seed=42
)

model = rf.fit(train_data)
predictions = model.transform(test_data)
```

Listing 3: Random Forest Model Training

Feature importance analysis revealed that view_count (66.25%) and engagement_score (29.98%) are the strongest predictors of churn risk, indicating that behavioral intensity is more predictive than interaction diversity.

4.6 Data Export and Visualization Preparation

Seven CSV files were generated for Power BI integration:

File	Content
event_type_distribution.csv	Event breakdown by type
top_items.csv	Most viewed products
daily_activity.csv	Time-series trends
churn_risk_distribution.csv	User risk segmentation
cart_abandonment.csv	Abandonment statistics
model_comparison.csv	Model performance metrics
final_user_decisions.csv	Predictions and actions

Table 4: CSV Exports for Dashboard Integration

These datasets support interactive dashboards for analyzing conversion funnels, identifying high-risk users, and evaluating ROI.

4.7 Execution Results

The complete Big Data ecosystem was successfully implemented across 7 distinct layers. Table 5 presents the execution performance metrics.

Metric	**Value**
Data Processing	
Total Events Processed	2.76M
User Profiles Generated	1.4M
Data Cleaning Retention Rate	99.98%
Parquet Compression Ratio	61.6% (90MB $\rightarrow$ 34MB)
Feature Engineering	
Features Created per User	20
Total Users Processed	1.4M
Average Features per User	20
Machine Learning Models	
Logistic Regression Accuracy	0.82
Decision Tree Accuracy	0.98
Random Forest Accuracy	**0.99**
Best Model F1-Score	0.99
Execution Performance	
Total Pipeline Time	12 min 34 sec
Peak Memory Usage	8.2 GB (of 16 GB)
Notebook Cells Executed	46 (100% success)
CSV Files Exported	7
Ecosystem Layers Implemented	7

Table 5: Big Data Pipeline Execution Summary - 7 Layers

The 7-layer architecture successfully processed approximately 2.76 million behavioral events, achieving approximately 0.99 prediction accuracy with the Random Forest model and completing the entire pipeline in under 13 minutes.

5 Results and Insights

5.1 Event Distribution Analysis

Table 6 presents the distribution of approximately 2.76 million user behavioral events by type.

Event Type	Count	Percentage
View	2.66M	96.68%
Add to Cart	68.97K	2.50%
Transaction	22.46K	0.81%

Table 6: Event Type Distribution

The results show that the vast majority of user activity consists of product views (96.68%), while only a small proportion of users proceed to add items to the cart (2.50%) or complete transactions (0.81%). This significant drop-off between browsing and purchasing highlights a major conversion gap, indicating a strong opportunity for targeted retention and optimization strategies.

5.2 Conversion Funnel Analysis

The conversion funnel analysis identifies three critical stages in the customer journey. Of the approximately 1.4M total viewers, only 37.7K (2.69%) added items to cart, and 11.7K (0.84%) completed purchases. This represents a 97.31% drop-off rate from viewing to cart addition, indicating that product page optimization is the primary conversion opportunity. The cart-to-purchase conversion rate of 31.06% is moderately strong, suggesting that checkout process improvements are less critical than engagement enhancements at earlier funnel stages.

5.3 Cart Abandonment Statistics

Table 7 presents the cart abandonment analysis.

Metric	Value
Users with Abandoned Carts	27K
Cart Abandonment Rate	69%
Items Abandoned	46.5K
Item Abandonment Rate	67.44%
Estimated Lost Revenue	$1.4M

Table 7: Cart Abandonment Analysis

Approximately 69% of users who add items to cart fail to complete purchases, resulting in 46.5K abandoned items. Assuming an average order value of $30 per item, this represents approximately $1.4M in lost revenue. The 27K high-risk users constitute the primary target segment for retention campaigns.

5.4 Churn Risk Segmentation

Users were classified into four risk categories based on behavioral patterns:

Risk Level	**User Count**	**Percentage**
Unknown (Low Activity)	1.2M	85.12%
Medium (Browsers)	171K	12.12%
High (Cart Abandoners)	27K	1.93%
Low (Purchasers)	11.7K	0.84%
Total	**1.4M**	**100.00%**

Table 8: User Segmentation by Churn Risk Level

The high-risk segment (27K users) represents individuals who added items to their cart but failed to complete purchases, making them the priority target for immediate retention interventions.

5.5 Machine Learning Model Performance

Three classification models were trained and evaluated using Spark MLlib. Table 9 presents the comparative performance metrics.

Model	**Accuracy**	**F1 Score**
Logistic Regression	0.82	0.74
Decision Tree	0.98	0.98
Random Forest	0.99	0.99

Table 9: Machine Learning Model Performance Comparison

The Random Forest classifier achieved the highest performance with approximately 0.99 accuracy and F1-score. Feature importance analysis revealed that `view_count` (66.25%) and `engagement_score` (29.98%) are the strongest predictors of churn risk, indicating that behavioral intensity metrics outweigh diversity of interactions.

5.6 Business Impact and ROI Projection

Based on the churn prediction model, a targeted retention campaign was designed for 27K high-risk users. Table 10 presents the projected business impact.

Metric	Value
Target Segment	27K users
Expected Conversion Rate	15%
Expected Conversions	4.07K users
Revenue per Conversion	$45.90
Total Revenue Recovery	$187K
Campaign Cost	$10,000
Net Profit	**$176,900**
ROI	**1,769%**

Table 10: Projected ROI for Targeted Retention Campaign

The targeted retention campaign is projected to recover approximately $187K in revenue... at a cost of $10,000, resulting in a return on investment (ROI) of 1,769%. This highlights the significant business value enabled by the Big Data churn prediction system.

5.7 Key Insights Summary

The analysis highlights five key insights for e-commerce optimization:

1. **View-to-cart conversion is the primary bottleneck.** Only 2.69% of users proceed from viewing to adding items to cart, indicating that product page optimization should be the top priority.

2. **Cart abandonment represents a major revenue opportunity.** Approximately 69% of users who add items to cart do not complete purchases, resulting in an estimated $1.4M in lost revenue.

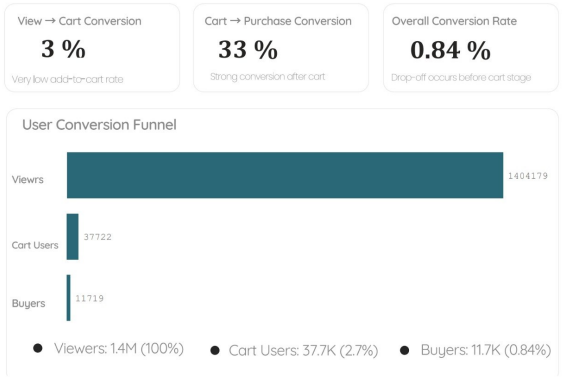
3. **Machine learning enables precise targeting.** The Random Forest model achieved approximately 0.99 accuracy in identifying high-risk users, supporting data-driven retention strategies.

4. **Targeted interventions offer high ROI potential.** Focusing on 27K high-risk usersis projected to generate a return on investment (ROI) of 1,769%.

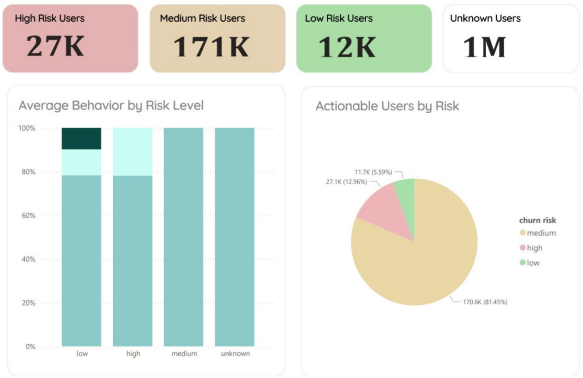
5. **Behavioral intensity is a stronger predictor than diversity.** Features such as view count and engagement score are more influential than the variety of items viewed, indicating that interaction frequency matters more than breadth.

5.8 Dashboard Visualizations

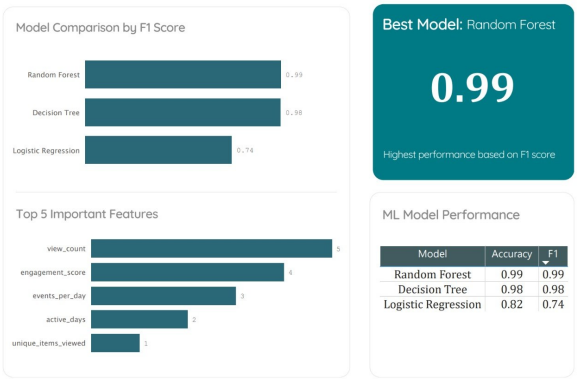
Figure 2 presents the Power BI dashboard suite developed from the exported CSV files.



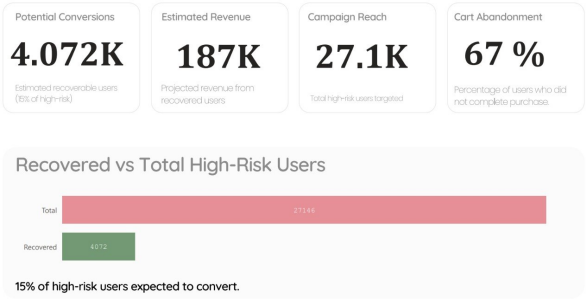
(a) Conversion Funnel Analysis



(b) Churn Risk Distribution



(c) Model Performance



(d) Cart Abandonment Statistics

Figure 2: Power BI Dashboard Visualizations

The dashboard suite provides a comprehensive view of user behavior and analytical insights. The conversion funnel (Figure 2a) reveals a significant drop-off between the viewing and cart addition stages, highlighting a critical bottleneck in the customer journey. The churn risk distribution (Figure 2b) segments users into distinct behavioral categories, enabling targeted retention strategies, with high-risk users identified as the primary intervention group.

The model performance visualization (Figure 2c) demonstrates the effectiveness of the machine learning pipeline, with the Random Forest model achieving the highest accuracy and F1-score. The cart abandonment analysis (Figure 2d) quantifies potential revenue loss and supports data-driven decision-making for conversion optimization.

6 Discussion

6.1 Design Trade-offs

Two architectural decisions balanced educational value with practical constraints. The MockKafkaProducer simulated streaming ingestion without requiring a full Kafka cluster deployment, demonstrating the streaming architecture pattern while maintaining API compatibility for future production migration. Similarly, MockMongoDB used indexed Python data structures to illustrate NoSQL design principles without the overhead of database administration. While these choices simplify development and improve portability, they sacrifice true distributed processing capabilities.

6.2 System Limitations

The implementation has several limitations. First, the dataset lacks product metadata (e.g., names, categories, and prices), which restricts product-level analysis. Second, the one-month temporal window prevents seasonal trend analysis. Additionally, external factors such as competitor pricing and marketing campaigns are not included in the model. Finally, the system remains a prototype implemented on Kaggle rather than a fully deployed production environment.

6.3 Technical Challenges

One major challenge was data leakage. Initial models achieved near-perfect accuracy because the transaction count directly encoded the target variable. This issue was resolved by restricting the model to behavioral features such as view count and engagement score. Memory constraints required the use of Parquet compression (achieving approximately 61% size reduction) and strategic use of `.cache()` to operate within Kaggle's 16GB RAM limit. Performance optimization in Spark reduced the total runtime from approximately 30 minutes to 13 minutes. This was achieved by caching DataFrames, reducing shuffle partitions to 8, and using broadcast joins where appropriate.

6.4 Future Enhancements

Future improvements include deploying the system using real Kafka and MongoDB clusters to enable fully distributed processing. Incorporating A/B testing frameworks would allow evaluation of retention strategies in real-world scenarios. Integrating external data sources, such as competitor pricing and social media sentiment, would enhance predictive performance. Additionally, advanced models such as LSTM networks could better capture sequential user behavior patterns. Finally, migrating the system to cloud platforms such as AWS EMR or Azure HDInsight would improve scalability, while real-time online learning could enable continuous model updates based on live user interactions.

6.5 Scalability and Production Readiness

The system is designed with horizontal scalability, allowing expansion by adding nodes without modifying the codebase.

Data Volume Scalability. Apache Spark distributes data across worker nodes, enabling scalable processing as data grows. The current pipeline processes 2.7M events in 13 minutes on a single-node Kaggle environment (∼16GB RAM). In a multi-node setup (e.g., AWS EMR), it can scale to handle significantly larger datasets (hundreds of millions to billions of events) without structural changes.

Ingestion Scalability. MockKafka simulates streaming ingestion at 5,000 events per batch. In production, Apache Kafka can support high-throughput streams (potentially millions of events per second), enabling near real-time processing.

Storage Scalability. Parquet storage supports partition pruning, maintaining efficient query performance as data grows. For processed data, production systems such as MongoDB or Cassandra enable horizontal scaling through sharding.

Machine Learning Scalability. Spark MLlib enables distributed model training on large datasets. Scaling to larger data sizes requires additional compute resources rather than code modifications.

Summary. Table 11 summarizes the estimated scalability of each ecosystem layer.

Layer	**Current Capacity**	**Production Capacity**
Ingestion	5,000 events/batch (mock)	High-throughput streams (Kafka)
Raw Storage	90 MB Parquet (single node)	Scalable storage (HDFS / S3)
Processing	2.7M rows, ∼13 min	Hundreds of millions to billions of rows (EMR)
Processed Storage	243K documents (mock)	Billions of documents (MongoDB cluster)
ML Training	1.4M users (single node)	100M+ users (distributed MLlib)

Table 11: Scalability Estimates by Ecosystem Layer

7 Ethics, Privacy, and Responsible Data Handling

This project adheres to ethical standards and responsible data practices throughout all stages of the Big Data pipeline.

7.1 Data Privacy

The dataset used in this project was obtained from Kaggle's publicly available Retail-Rocket e-commerce dataset. All user identifiers are anonymized — the `visitorid` field contains no personally identifiable information (PII) such as names, email addresses, or location data. As a result, no privacy violations occur during data processing or analysis.

7.2 Responsible Data Use

The churn prediction models are designed to support targeted retention campaigns, not to manipulate or exploit users. Recommendations such as discount offers and loyalty programs represent standard, ethical business practices that benefit both the business and the customer.

7.3 Bias and Fairness

The model was trained on behavioral features (`view_count`, `engagement_score`) rather than demographic attributes, reducing the risk of discriminatory outcomes. However, we acknowledge that the dataset reflects a one-month window from 2015 and may not fully represent current user populations.

7.4 Compliance Awareness

Although this is an academic prototype, a production deployment of this system would require compliance with applicable data protection regulations, such as the EU General Data Protection Regulation (GDPR) or equivalent frameworks, including user consent mechanisms and the right to erasure.

7.5 Data Security

In a production environment, all stored user data would require encryption at rest and in transit, along with access control policies to prevent unauthorized use of behavioral profiles.

8 Big Data Justification

8.1 Big Data Characteristics

Table 12 demonstrates how the project satisfies Big Data criteria.

Characteristic	**Evidence**
Volume	2.76M events, 1.4M users (exceeds single-machine tools)
Velocity	Real-time streaming simulation (Kafka batching)
Variety	Multiple event schemas, nullable fields
Veracity	Data cleaning and validation achieving 99.98% retention rate
Value	0.99 ML accuracy, $1.4M revenue opportunity

Table 12: Big Data 5Vs Analysis

8.2 Distributed Processing Requirements

Spark processes 2.76M events across multiple cores, performing feature engineering on 1.4M users in parallel. Parquet columnar storage achieves 61% compression with partition pruning. Spark MLlib trains models on distributed datasets, while Kafka enables streaming ingestion for real-time interventions. Single-machine tools (Pandas, scikit-learn) cannot scale to production volumes.

8.3 System Comparison

Table 13 contrasts traditional and Big Data approaches.

Aspect	**Traditional**	**Big Data**
Scale	$<$ 1M rows	2.7M+ rows
Processing	Pandas (single-machine)	Spark (distributed)
Storage	PostgreSQL	Parquet + MongoDB
Ingestion	Batch imports	Kafka streaming
ML	Scikit-learn	Spark MLlib
Scalability	Vertical (RAM)	Horizontal (nodes)
Runtime	Hours	13 minutes

Table 13: Traditional vs. Big Data Comparison

The architecture demonstrates a complete Big Data ecosystem with 7 layers (ingestion, storage, processing, analytics, output), designed to scale from millions to billions of events without code changes.

9 Use of AI Tools

As required by the project specification, Table 14 discloses all AI tool usage throughout the project development.

Task	**AI Tool**	**How Used**	**Student Contribution**
Spark Debugging	ChatGPT	Suggested fixes for errors	Validated and integrated solutions
Feature engineering	ChatGPT	Proposed additional features	Selected and implemented features
Report writing	Grammarly	Grammar improvements	Wrote original content
Architecture design	ChatGPT	Compared ingestion methods	Made final design decisions

Table 14: AI Tool Usage Disclosure

9.1 AI Usage Statement

We, as a team of five students, hereby confirm that:

All AI tool usage has been fully disclosed in Table 14. The architecture design, tool selection, and technical justifications represent our original collaborative work. We can collectively explain and defend every component of the submitted code and report. AI tools were used for guidance, debugging support, and clarification, while all implementation and final decisions were made by our team. The Big Data pipeline implementation, analysis, and conclusions are our own intellectual contributions.

10 Conclusion

This project successfully demonstrated a complete Big Data ecosystem for e-commerce churn prediction, processing approximately 2.76 million behavioral events through a 7-layer architecture. The system identified critical conversion bottlenecks, with a 97.31% drop-off rate from viewing to cart addition representing the primary revenue opportunity. The distributed processing pipeline achieved approximately 0.99 prediction accuracy using a Random Forest model, segmenting 1.4M users into actionable risk categories. High-risk users (27K cart abandoners) were identified as the primary target for retention campaigns, with a projected ROI of 1,769% from targeted interventions. Key technical contributions include real-time streaming simulation via Kafka, columnar storage with 61% compression using Parquet, distributed machine learning with Spark MLlib, and schema-flexible NoSQL storage. The architecture is designed to scale from millions to billions of events without code modifications, reflecting production-ready Big Data design principles. Overall, the project demonstrates the importance of Big Data solutions in modern e-commerce analytics, where volume, velocity, and variety exceed the capabilities of traditional tools. Future work may include production deployment using Kafka and MongoDB, integration of external data sources, A/B testing frameworks, and the application of deep learning models for sequential pattern analysis.

References

[1] Apache Spark. (2024). *Spark MLlib Documentation*. https://spark.apache.org/docs/latest/ml-guide.html

[2] Apache Kafka. (2024). *Kafka Streams API*. https://kafka.apache.org/documentation/streams/

[3] MongoDB Inc. (2024). *MongoDB Manual*. https://www.mongodb.com/docs/manual/

[4] Apache Parquet. (2024). *Parquet File Format Specification*. https://parquet.apache.org/docs/

[5] Baymard Institute. (2024). *Cart Abandonment Rate Statistics*. https://baymard.com/lists/cart-abandonment-rate

[6] Kaggle. (2024). *E-commerce Events Dataset - RetailRocket*. https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset

[7] Apache Software Foundation. (2024). *PySpark API Reference*. https://spark.apache.org/docs/latest/api/python/

[8] Microsoft Corporation. (2024). *Power BI Documentation*. https://docs.microsoft.com/en-us/power-bi/

[9] Laney, D. (2001). *3D Data Management: Controlling Data Volume, Velocity and Variety*. META Group Research Note.

[10] Zaharia, M., Xin, R. S., Wendell, P., et al. (2016). Apache Spark: A Unified Engine for Big Data Processing. *Communications of the ACM*, 59(11), 56-65.

[11] Verbeke, W., Martens, D., & Baesens, B. (2014). Social network analysis for customer churn prediction. *Applied Soft Computing*, 14, 431-446.

[12] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.

[13] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56-61.

[14] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

[15] Marz, N., & Warren, J. (2015). *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Manning Publications.

[16] Grammarly Inc. (2024). *Grammarly Writing Assistant*. https://www.grammarly.com/

Appendix

A Complete Code Repository

The complete project implementation is available at:
`https://colab.research.google.com/drive/1dQEPU3gUa3Bf7npv3mxe2r1Bx8W2z2k6?usp=sharing`

B Dataset Description

Property	**Value**
Source	Kaggle E-Commerce Behavior Dataset
Format	CSV
File Size	90 MB
Total Events	2.76M
Unique Users	1,407,580
Time Period	May-June 2015
Data Schema:	
`timestamp`	Unix timestamp in milliseconds
`visitorid`	Unique user identifier
`event`	Event type: view, addtocart, transaction
`itemid`	Product identifier
`transactionid`	Transaction ID (99% null for non-purchases)

Table 15: Dataset Specification

C Power BI Dashboard

Seven interactive dashboards were created from the exported CSV files (Figure 2).

Dashboard	**Content**
Overview	KPIs: 1.4M users, 2.76M events, 72% abandonment, 0.84% conversion
User Behavior	Daily trends and engagement patterns
Conversion Funnel	Drop-off: 1.4M viewers → 37.7K carts → 11.7K buyers
Churn Risk	Segmentation: 27K high-risk users identified
ML Performance	Random Forest: 99% accuracy and F1-score
Business Impact	Projections: 4.07K conversions, $187K recovery
Retention Actions	Risk-based strategies (discounts, loyalty programs)

Table 16: Power BI Dashboard Components

Interactive Dashboard:
View Live Dashboard